RoboGuardian: Fault-Aware Execution and Memory Maintenance for Embodied Agent Harness Frameworks

Anonymous Author(s)

Abstract

Embodied-agent harnesses connect high-level planners to low-level execution policies and use execution results to update task state and memory. However, current harness mechanisms often fail to verify physical state changes reliably. This limitation applies to both low-level execution feedback and VLM-based evaluations. Unverified outcomes may therefore enter memory and mislead subsequent actions. In this paper, we present **RoboGuardian**, a fault-aware execution and memory-maintenance framework for embodied-agent harnesses. It distinguishes outdated plan steps, uncertain outcomes, and execution failures by their required recovery strategies. This categorization enables robust decision-making from multi-modal evidence. RoboGuardian checks plan validity and entity identity before execution, combining semantic, proprioceptive, and object-level perception to produce a three-way physical judgment where only physically confirmed actions update its embodied memory and progress. It also maintains a bounded trusted context and re-observes before retrying unresolved actions, preventing blind retry and replay. We evaluate RoboGuardian across diverse simulation environments, including robotic arm manipulation on LIBERO-10, dexterous grasping on Clutter-Bench, and continuous vision-language navigation in Habitat. Furthermore, we deploy the system in the real world on both dual-arm and dexterous-hand manipulation platforms. Experimental results demonstrate that RoboGuardian effectively enhances execution reliability across diverse tasks and hardware embodiments. The framework maintains a streamlined and effective memory mechanism while introducing minimal computational overhead during execution.

CCS Concepts

• **Computing methodologies** → **Intelligent agents**; Robotic planning; • **Computer systems organization** → *Robotic autonomy*.

Keywords

embodied agents, agent harness, execution fault detection, failure recovery, robot memory

1 Introduction

Advances in learned visuomotor policies and vision-language-action models have enabled general-purpose language-conditioned manipulation [2, 5, 17, 40, 43]. In parallel, language-model planners compose and invoke motor skills [1, 13, 22]. To integrate language-model planning with perception, actuation, and heterogeneous hardware, embodied systems rely on harness architectures that coordinate these components through a unified interface [3, 25, 35]. Existing architectures commonly follow a "Plan and Execute" paradigm, in which an agent interprets an instruction, a planner converts it into motor sequences, and hardware executes the corresponding skills. However, these systems remain incomplete. By relying only on this forward execution path, existing harnesses lack dedicated mechanisms for rigorous post-execution verification and memory maintenance.

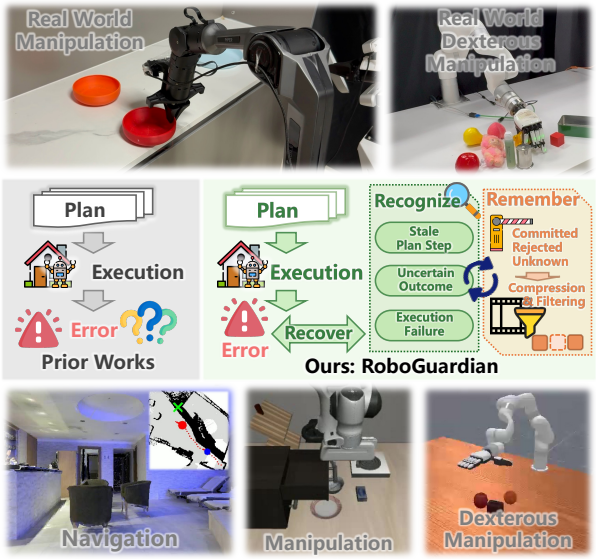


Figure 1: Prior embodied-agent harnesses may write unverified runtime results into task state and memory, allowing errors to propagate. RoboGuardian checks actions before dispatch, verifies physical effects from multimodal evidence, gates memory updates, and re-observes the environment before retrying, enabling robust execution across diverse robot embodiments.

Existing works attempt to augment execution through vision-language success and failure detection [8, 9], experience summaries [10, 27], visual monitors [20, 42], or explicit memory managers [14, 29, 36, 39, 41]. However, confined to a basic Plan and Execute loop, existing systems lack unified rules for action dispatch, memory updates, and retries. Robust execution must overcome three fundamental challenges. First, software signals often contradict complex physical realities, masking the true environmental state. Second, indiscriminately writing unverified results into memory propagates errors that derail long-horizon tasks. Third, generic retry strategies are inadequate and potentially dangerous. Distinct exceptions, such as outdated plans or execution failures, demand specific, tailored recoveries.

To address these challenges and improve system robustness accordingly, we propose **RoboGuardian**, a framework for fault-aware execution and memory maintenance in embodied-agent harnesses. As shown in Fig. 1, RoboGuardian expands the basic harness architecture into four connected stages. While maintaining the standard *Plan and Execute* logic for task understanding and action proposal, we focus on three critical post-execution capabilities

designed to explicitly resolve the aforementioned issues: (1) Execution result recognition and classification (***Recognize*****):** treating software results as untrusted and combining multimodal evidence to verify and classify physical effects, thereby overcoming the limits of binary failure states; (2) Lightweight memory updating and maintenance (***Remember*****):** utilizing a commit gate to ensure that only strictly verified outcomes update the agent's memory, halting the propagation of errors; and (3) Failure-category-based retry (***Recovery*****):** deploying tailored corrective strategies—such as replanning, alternative skill selection, or re-observation—matched specifically to the recognized error category to replace blind retries. We take the planner and skill library as given, focusing entirely on the critical authority boundary between a skill returning and the planner acting on that return.

To mitigate categorized errors and prevent their propagation, we introduce an *Effect Receipt* mechanism coupled with a gated memory system. In embodied tasks, many physical skills are non-idempotent. Blind retries under uncertain outcomes risk duplicating actions, making them riskier than pausing execution. The Receipt mechanism mandates re-observation before retrying and ensures that only COMMITTED receipts enter the trusted memory state. REJECTED and UNKNOWN attempts are isolated into a separate Attempt Log for recovery diagnosis, preventing them from being mistaken as task progress. By organizing verified facts, verified progress, and relation memory into a bounded context, this design limits the long-term impact of execution errors and controls context costs during long-horizon tasks.

Evaluating such a harness requires more than final task success. Consider a rollout in which a skill response is lost, the system repeats a non-idempotent action, and the final environment happens to satisfy the task condition. A conventional metric records success but does not capture the duplicate action or incorrect memory. We therefore report verified success rate in addition to task success rate, together with unverified memory writes, duplicate physical actions, recovery coverage, recovery success, and context cost. These metrics measure the final task outcome, the reliability of the execution process, and the cost of operating the harness.

In summary, our contributions are as follows:

- We analyze how embodied agent harnesses misjudge skill outcomes and derive a compact taxonomy of outdated plan steps, uncertain outcomes, and execution failures. The categories are defined by their required responses rather than their physical root causes, making them decidable from evidence available at decision time.
- We introduce RoboGuardian, which integrates action admission, multimodal physical-effect verification, effect-receipt-gated memory, and typed recovery into a unified pipeline. It prevents physical retries under insufficient evidence and controls memory size through a bounded context.
- We evaluate RoboGuardian in simulation on LIBERO-10 [24], dexterous grasping on Clutter-Bench [4], and continuous vision-language navigation in Habitat [6, 19, 34]. We further validate it on two realworld physical platforms, a dexterous hand and a gripper-based manipulator, and analyze the roles of fault recovery and memory maintenance.

Table 1: Functional comparison with representative methods, grouped by the three related-work areas in Sec. 2.1–2.3. AG denotes action-admission gating. MA denotes multimodal assessment. EA denotes error-category attribution. TR denotes typed recovery feedback. RO denotes re-observation before retry. VM denotes verified-only memory. BM denotes bounded memory. ✓ marks a design target. ▲ marks partial support. ✗ marks no central mechanism.

	Plan & Execute	*Recognize*		*Recovery*		*Remember*	
Method	AG	MA	EA	TR	RO	VM	BM
Sec. 2.1: Harnesses and skill invocation							
SayCan [1]	▲	✗	✗	✗	✗	✗	✗
Code as Policies [22]	✗	✗	✗	✗	▲	✗	✗
SayPlan [31]	▲	✗	✗	▲	✗	✗	▲
GUAVA [25]	▲	✓	✗	▲	✗	✗	✗
RoboOS [35]	▲	▲	✗	▲	✗	✗	✗
ROSClaw [3]	✓	▲	✗	▲	✗	✗	▲
MoMaStage [20]	✓	▲	✗	▲	✗	✗	✗
Sec. 2.2: Execution monitoring							
Inner Monologue [13]	✗	▲	✗	✗	▲	✗	✗
DoReMi [10]	▲	▲	✗	▲	▲	✗	✗
REFLECT [27]	✗	✓	▲	▲	✗	✗	✗
AHA [9]	✗	▲	✓	✗	✗	✗	✗
Code-as-Monitor [42]	✓	✓	✗	▲	✗	✗	✗
Sec. 2.3: Agent memory							
Reflexion [32]	✗	✗	✗	▲	✗	✗	▲
Voyager [36]	✗	✗	✗	▲	✗	▲	▲
MemGPT [29]	✗	✗	✗	✗	✗	✗	✓
VLA memory [14]	✗	▲	✗	✗	✗	▲	▲
RoboGuardian (ours)	✓	✓	✓	✓	✓	✓	✓

2 Related Work

To contextualize our approach, Table 1 compares the mechanisms of action admission, outcome recognition, recovery, and memory across three primary areas of related work. As detailed in the following subsections, while embodied harness frameworks (Sec. 2.1) effectively constrain what skills may be invoked, they rarely define when a returned outcome becomes a verified physical fact. Similarly, execution monitors (Sec. 2.2) detect or explain abnormal execution, but their outputs typically remain advisory signals for replanning rather than strict execution gates. Finally, agent memory systems (Sec. 2.3) preserve and retrieve extended context, yet they generally assume that the stored outcomes are inherently reliable. RoboGuardian addresses these gaps by integrating physical-effect verification directly into the memory and recovery pipeline.

2.1 Embodied Agent Harness Frameworks

Recent embodied systems often use a language model to select and invoke parameterized robot skills. Language models can generate actionable plans without task-specific training [11]. SayCan [1] grounds such plans with learned affordance values. Later work generates executable programs over perception and control primitives [22]. Other methods add feasibility checks [23], use 3D scene graphs for large environments [31], or integrate perception into the planning model [7]. Other frameworks support long-horizon execution across heterogeneous robot embodiments. GUAVA [25] studies the action, observation, and workflow design of embodied tool use. RoboOS [35] coordinates heterogeneous robots and skills through a hierarchical architecture. RoboClaw, ROSClaw, and ABot-Claw explore long-horizon orchestration, ROS 2 execution, and cross-embodiment collaboration [3, 15, 21]. MoMaStage [20]

uses a skill-state graph to constrain planning and closed-loop execution. Robot skills have also advanced from imitation and diffusion policies [5, 40] to vision-language-action models [2, 17, 33, 43]. ClawKeeper [26] focuses on software and system security for OpenClaw agents, which differs from physical-effect verification.

These systems improve which skills are selected, constrained, and coordinated. They do not centrally define what the harness may conclude after a skill returns. A single runtime result can hide an outdated step, a lost response, or a missing physical effect. Forwarding it directly can trigger duplicate actions or store false prerequisites for later skills.

2.2 Execution Failure Detection and Recovery

Another line of work adds explicit execution monitoring. These methods construct different implicit verifiers. Inner Monologue [13] uses textual success detection and scene descriptions as feedback to the planner. Grounded Decoding [12] constrains generation with grounded model likelihoods. DoReMi [10] detects plan-execution misalignment and triggers recovery. Code-as-Monitor [42] compiles spatiotemporal constraints into visual programs for proactive and reactive monitoring. REFLECT [27] summarizes multimodal experience and uses a language model to explain failures and guide corrections. AHA [9] trains a vision-language model to detect and reason about manipulation failures. Some work also studies VLM-based success detection [8] and explicit state tracking [39].

These methods show the value of execution feedback, but their outputs are often success decisions, constraint violations, or language explanations. They rarely assign an exception to a category with a distinct authority consequence. A failed grasp, should be rejected before dispatch if the action is outdated, re-observed before retry if the response was lost, and rejected at commit if independent evidence shows no grasp effect. A monitor that only reports failure does not determine which authority decision is safe. Without a gate on task state and memory, an incorrect success can become a trusted prerequisite and be retrieved repeatedly by later steps.

2.3 Memory in Embodied-Agent Harnesses

Agent memory research mainly studies what information to store, how to update it, and how to retrieve it. Representative methods record interleaved reasoning and action trajectories [38], retain verbal self-reflection across attempts [32], or summarize and retrieve observation streams [30]. Other approaches use tiered storage to extend limited context windows [29], introduce forgetting mechanisms [41], or accumulate reusable skills through continual exploration [36]. Long-term memory also supports open-world execution for vision-language-action agents [14]. These methods improve memory content, management, and retrieval, but generally assume that recorded observations and action outcomes are reliable. This assumption is fragile in embodied execution, where a software result may not reflect the actual physical effect. Better retrieval cannot turn an unverified claim about the physical state into a reliable fact. It can only make that claim more influential in subsequent decisions.

3 Method

In this work, we introduce RoboGuardian as a fault-aware execution and memory-maintenance framework for embodied-agent harnesses. It specifically focuses on classifying execution errors by their sources, performing lightweight memory updates, and deploying tailored recovery strategies. The framework does not treat software-level success as a physical fact. It checks proposed actions before dispatch, verifies their physical effects after execution, and commits only verified effects. Unresolved outcomes require new evidence before any retry. Fig. 2 shows the four-stage pipeline. We take the planner and skill library as given.

3.1 Problem Formulation

We assume that a language-guided embodied agent operates over a set of robot embodiments $\mathcal{E}$. Each embodiment $b \in \mathcal{E}$ has a skill library $\mathcal{K}_b$, an action space, and an embodiment state. Given a language instruction ℓ and the current trusted context $\mathcal{B}_t$, the planner selects an embodiment and proposes an action:

$$a_t = \pi_{\text{plan}}(\ell, \mathcal{B}_t) = (b_t, k_t, \theta_t), \tag{1}$$

where $b_t \in \mathcal{E}$ is the selected embodiment and $k_t \in \mathcal{K}_{b_t}$ is one of its skills. The parameters θ_t specify elements such as the target object and target pose. The robot runtime executes a_t and returns a software status r_t. The system also receives observations before and after execution. We denote them by o_{t-1} and o_t, with

$$o_t = (\mathcal{I}_t, q_t^{b_t}), \tag{2}$$

where $\mathcal{I}_t$ is a visual observation and $q_t^{b_t}$ is the current embodiment state. The status r_t describes the software execution path. It does not prove that the intended physical effect occurred. A successful gripper command does not guarantee a stable grasp. A terminated navigation command does not guarantee that the base lies within the goal tolerance. The system therefore cannot use r_t alone to update task progress or trusted memory. We verify the physical effect from the action, software status, and observations:

$$z_t = \Phi(a_t, r_t, o_{t-1}, o_t) \in \{\text{yes}, \text{no}, \text{uncertain}\}. \tag{3}$$

The judgment yes confirms the intended effect. The judgment no confirms the effect is absent. The judgment uncertain indicates insufficient evidence. Only verified effects may update state. An uncertain outcome must be re-observed before any physical retry.

3.2 Plan and Execute with Admission-Gated Action Dispatch

A proposed action is not dispatched immediately. The admission module checks plan validity, skill applicability, and entity identity:

$$d_t = \mathcal{A}(a_t, \mathcal{B}_t, o_{t-1}) \in \{\text{dispatch}, \text{reject}\}. \tag{4}$$

The system treats a proposal as an outdated plan step when its preconditions, skill applicability, or entity identity no longer match the current state. It rejects the proposal before dispatch and returns typed feedback for replanning or policy re-inference from the latest observation. The identity check covers the target object, selected embodiment, and skill parameters. It prevents an old plan from acting on a new scene instance. It also prevents dispatch to an incompatible embodiment. The action enters the robot runtime only when d_t = dispatch. The runtime may report completion,

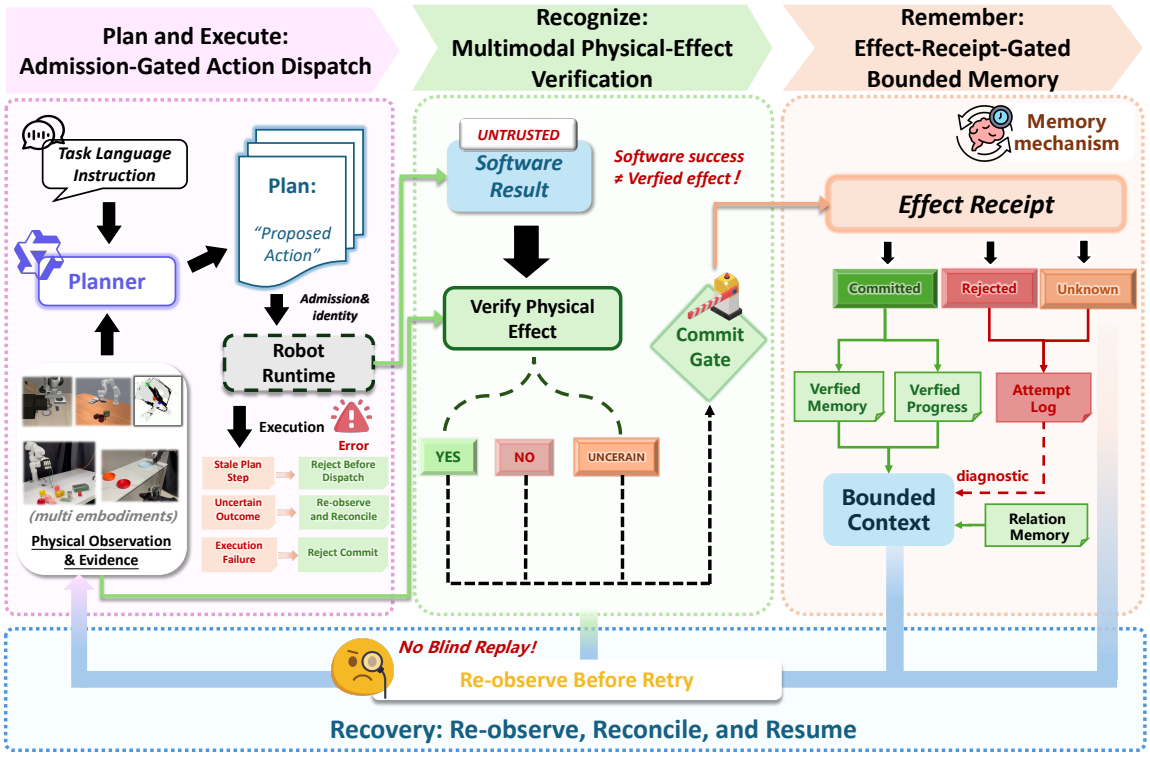


Figure 2: RoboGuardian provides fault-aware execution and memory maintenance across heterogeneous robot embodiments.

timeout, or a low-level error. RoboGuardian retains these signals as evidence but does not treat them as physical facts. Recognize evaluates every dispatched action. The three exception categories are assigned by their required responses. An outdated plan step is rejected before dispatch. An uncertain outcome arises when z_t = uncertain and triggers re-observation. An execution failure is assigned only when z_t = no and blocks positive commitment.

3.3 Recognize with Multimodal Physical-Effect Verification

Recognize determines whether a dispatched action produced its intended physical effect. No single source provides sufficient evidence in every setting. Visual observations can be occluded. Proprioception cannot fully describe environmental change. Effect measurements may not distinguish an unattempted action from a failed one or require simulator state, task evaluators, or human labels. We therefore organize assessment around semantic, proprioceptive, and effect-level evidence families, while allowing each adapter to expose only the subset it can support. When visual-language verification is available, the semantic channel sends the post-execution image and an effect description to a vision-language model:

$$\phi_{\text{sem}} = f_{\text{VLM}}(\mathcal{I}_t, \textsc{Describe}(b_t, k_t, \theta_t)) \in \{\text{pass}, \text{fail}, \text{indet}\}. \quad (5)$$

The function Describe converts the action into a verification question. It may ask whether a mug is held by the gripper or whether the robot reached the goal region. The assessor returns indet when the target is occluded or outside the field of view. We preserve this abstention as uncertain evidence. The proprioceptive channel checks the skill-specific pre-state and post-state sets:

$$\phi_{\text{pre}} = \mathbb{1}\left[q_{t-1}^{b_t} \in Q_{b_t,k_t}^{\text{pre}}\right], \qquad \phi_{\text{post}} = \mathbb{1}\left[q_t^{b_t} \in Q_{b_t,k_t}^{\text{post}}\right]. \quad (6)$$

A violated precondition indicates that the action may no longer be applicable. A violated postcondition indicates that the embodiment did not reach the required terminal state. The state variables depend on the embodiment. We use finger joints and palm pose for a dexterous hand. We use gripper width and end-effector pose for a gripper-based manipulator. We use base pose and velocity for a mobile robot. The effect-level channel compares the expected target transformation $\Delta\hat{T}_t$ with the measured transformation ΔT_t:

$$\phi_{\text{eff}} = \mathbb{1}\left[d_{\text{p}}(\Delta T_t, \Delta\hat{T}_t) < \epsilon_{\text{p}}^{b_t,k_t} \ \wedge \ d_{\text{r}}(\Delta T_t, \Delta\hat{T}_t) < \epsilon_{\text{r}}^{b_t,k_t}\right]. \quad (7)$$

The functions d_{p} and d_{r} measure translational and rotational discrepancies. Their thresholds depend on the embodiment and skill class. The system also records a partial-effect signal $\phi_{\text{eff}}^{\text{part}}$. This signal indicates motion in the intended direction without goal completion. It contributes to fusion but cannot update completed progress.

The fusion module collects the runtime status and all evidence channels in a structured report:

$$\rho_t = \left\langle r_t, \phi_{\text{sem}}, \phi_{\text{pre}}, \phi_{\text{post}}, \phi_{\text{eff}}, \phi_{\text{eff}}^{\text{part}} \right\rangle. \quad (8)$$

This fusion step instantiates the verifier Φ from Eq. (3) by combining ρ_t with the current observation and subgoal g_t:

$$z_t = f_{\text{fuse}}(\rho_t, o_t, g_t). \quad (9)$$

The closed judgment set gives each outcome a defined transition. The Commit Gate converts the physical judgment into an Effect

Receipt:

$$e_t = \mathcal{G}(z_t) = \begin{cases} \text{committed}, & z_t = \text{yes}, \\ \text{rejected}, & z_t = \text{no}, \\ \text{unknown}, & z_t = \text{uncertain}. \end{cases} \tag{10}$$

The receipt stores the action identity, software status, key evidence, and physical judgment. It is the only interface that can authorize trusted memory updates.

3.4 Remember with Receipt-Gated Bounded Memory

The memory mechanism contains verified memory, verified progress, an attempt log, and relation memory. Only e_t = committed updates verified world facts and task progress. A rejected or unknown receipt leaves trusted state unchanged. The system records the action, evidence, receipt type, and attempt count in the attempt log. A rejected receipt means that the intended effect is confirmed absent. An unknown receipt means that the outcome remains unresolved. Relation memory stores object relations, skill relations, and task constraints when such relations are available. The system combines these relations with verified facts, verified progress, and typed diagnostics to form the bounded context:

$$\mathcal{B}_{t+1} = C\big(\mathcal{M}^{\text{ver}}_{t+1}, \mathcal{P}^{\text{ver}}_{t+1}, \mathcal{R}_{t+1}, \mathcal{D}(\mathcal{L}_{t+1})\big)\,. \tag{11}$$

Here, $\mathcal{L}$ denotes the attempt log and $\mathcal{D}$ extracts diagnostic summaries. Diagnostics help the planner avoid ineffective strategies. They cannot be promoted to physical facts. We impose a budget B on $\mathcal{B}_t$. The bounded view preserves recent observations, committed progress, and selected receipt-linked evidence, while the complete audit information remains outside the planner context in the attempt log.

3.5 Recovery through Re-observation and Reconciliation

Recovery selects the next procedure from the admission result, effect receipt, attempt log, and bounded context. Every path follows the same rule. A physical retry requires a fresh observation. For an outdated plan step, the system rejects dispatch and regenerates the plan suffix while preserving the committed prefix. For a rejected receipt, the system may choose an alternative skill, replan, or perform a bounded safe retry. The attempt log prevents repeated use of a strategy that has already failed. For an unknown receipt, the system executes no new task action solely from the unresolved outcome. It first obtains fresh evidence from the adapter, such as a new camera frame, evaluator state, or operator effect label. After receiving a new observation o'_t, the system forms a fresh report ρ'_t and recomputes $z'_t = f_{\text{fuse}}(\rho'_t, o'_t, g_t)$.

A yes result produces a committed receipt and resumes the task. A no result produces a rejected receipt and starts safe recovery. If uncertainty remains, the system gathers more evidence. After N_{obs} attempts, it marks the outcome as unresolved and requests stronger perception or human confirmation. A non-idempotent skill is never repeated directly from an unknown state. This rule prevents duplicate grasping, pushing, and placing effects.

Table 2: Evaluation metrics and their definitions.

Metric	Definition
Task Success Rate (TSR) ↑	Upstream task-evaluator judgment reported without modification.
Verified Success Rate (VSR) ↑	Rate of task-successful episodes whose execution chain has no outdated plan-step dispatch, duplicate dispatch, or unresolved outcome.
TSR–VSR Gap ↓	Declared successes that the harness cannot support with its own evidence.
Unverified Memory Writes ↓	Outcomes admitted to trusted memory without a verified physical effect.
Duplicate Physical Actions ↓	Logical invocations that produce more than one physical dispatch.
Recovery Coverage ↑	Rate of eligible episodes that enter recovery.
Recovery Success ↑	Rate of recovery cases that are resolved.
Context and Execution Cost ↓	Actions, model calls, wall-clock time, max images per request, and peak request size.

3.6 Implementation Details

All embodiments use the same pipeline for planning, verification, receipt gating, memory maintenance, and recovery. Components tied to the physical platform remain embodiment-specific. These include the skill library, state constraints, effect measurements, and observation tools. In simulation [4, 24, 34], the verifier directly reads object, end-effector, and robot-base poses from the environment. For dexterous hands, the state also includes finger joints, contact states, and palm pose. For gripper-based manipulators, it includes gripper width, contact force, and end-effector pose. For mobile robots, it includes base pose, goal distance, and residual velocity. The model configuration follows the native setup of each benchmark. Clutter-Bench uses Qwen3-VL-32B-Instruct [37] with a temperature of 0.2, an 800-token output limit, and 512×512 rendered PNG frames with `low` image detail. We allow up to three provider retries and set the planning budget to 60 turns. LIBERO uses a frozen SmolVLA [33] checkpoint as its action policy rather than an API-based VLM planner. The navigation experiments retain their native macro-level stack. GPT-4o [16] serves as the language-action model, while Qwen3.5-27B [37] serves as the vision-action model with reasoning output disabled.

For real-world execution, vision and robot sensors jointly estimate physical effects. DINOv2 features [28] provide target point prompts, and SAM [18] extracts object masks. Depth observations are then back-projected to estimate rigid transformations between consecutive point clouds. Wider effect tolerances account for perception noise. Within each benchmark, all variants share the same planner or policy, seeds, initial states, and interaction budget. Retry limits, re-observation limits, memory budgets, effect thresholds, and embodiment constraints are selected on validation data and fixed within each embodiment class.

4 Experiments

Our experiments evaluate RoboGuardian through four questions. **Q1:** Does RoboGuardian preserve task capability under fault-free execution? **Q2:** Can it prevent unverified updates and duplicate actions under execution exceptions? **Q3:** Can receipt-gated memory preserve verified progress while bounding context and execution cost? **Q4:** Does the framework transfer across action granularities, task modalities, robot embodiments, and physical platforms?

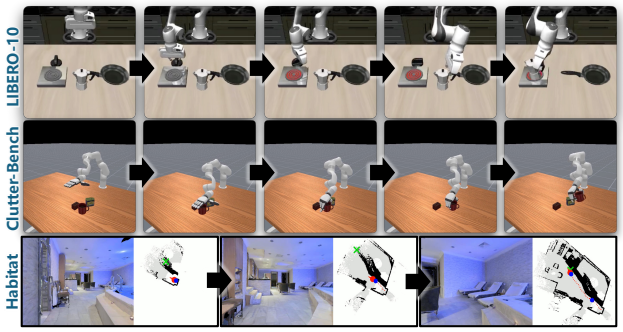


Figure 3: Experimental settings on the three simulated platforms.

4.1 Experimental Setup

We evaluate RoboGuardian in three simulated settings and on two physical robot platforms. The simulated settings include LIBERO-10 [24] for low-level manipulation, Clutter-Bench [4] for skill-level dexterous grasping, and LaViRA [6] in Habitat [34] for vision-language navigation. Clutter-Bench is a ManiSkill-based simulation introduced in AdaClearGrasp [4]. The physical platforms include an xArm manipulator equipped with an XHand dexterous hand for tabletop grasping and an ALOHA dual-arm platform with grippers for sequential manipulation. Fig. 3 summarizes the three simulated settings. On LIBERO-10, a frozen SmolVLA [33] policy produces low-level actions. This setting addresses Q1 and Q2 through action-level verification of stale plan steps and uncertain outcomes. Clutter-Bench is our primary quantitative setting and addresses Q1 through Q3 through skill-level verification, memory ablation, and context analysis. The macro-level LaViRA integration examines the cross-modal transfer in Q4. The physical deployments provide qualitative evidence for Q2 and Q4 using perception-derived verification and naturally occurring faults. Together, these settings span low-level actions, high-level skills, navigation episodes, and real-world execution. Because their evidence sources and integration granularities differ, we state the supported claims separately.

Variants. We use three comparison roles across the evaluation, although not every platform includes all three. **Base** denotes the original or ungoverned harness. It writes runtime returns directly into task state and memory. **Ours-Window** denotes the bounded-window variant used in the Clutter-Bench memory ablation. It gives the planner only the three most recent images and does not use receipt-gated memory. **RoboGuardian** maintains receipt-derived task progress and a bounded evidence context. It can retrieve at most two additional evidence frames linked to the current receipt. Within each platform, we match the planner, skills, policy checkpoint, model, temperature, interaction budget, random seeds, initial states, and fault schedule whenever they apply. The Clutter-Bench study additionally includes Original Ada, Window-3, CGEM, and the complete RoboGuardian system.

Metrics. Table 2 summarizes the metrics used across the evaluation. VSR is the primary metric in the action-level and skill-level quantitative studies. Platform-specific sections report only the metrics supported by their integration granularity. Arrows indicate preferred directions.

Table 3: LIBERO-10 results with SmolVLA. Each row contains 100 episodes. Actions and wall-clock time are per-episode means and serve as descriptive cost measures.

Variant	Condition	TSR ↑	VSR ↑	Actions ↓	Wall (s) ↓
Base	clean	0.43	0.43	418.6	358.4
RoboGuardian	clean	**0.42**	**0.42**	**395.7**	**349.2**
Base	outdated chunk	0.41	0.20	426.8	367.0
RoboGuardian	outdated chunk	**0.41**	**0.41**	**378.5**	**338.9**
Base	timeout/retry	0.42	0.22	420.4	374.3
RoboGuardian	timeout/retry	**0.39**	**0.39**	**387.8**	**354.9**

Fault categories. The experiments cover the three response-oriented exception categories defined in Section 3. An **outdated plan step** applies an action computed from an earlier observation to the current state. An **uncertain outcome** arises when the available evidence cannot determine whether an action took effect. An **execution failure** is assigned when physical verification returns no. Outdated action chunks in LIBERO instantiate outdated plan steps. Timeout and retry faults in LIBERO and response loss in Clutter-Bench instantiate uncertain outcomes. False software success in Clutter-Bench instantiates execution failure. The clean condition means that we inject no external fault. It does not imply that every policy or skill execution succeeds. We report execution stalls separately as a capability boundary rather than a primary exception category.

4.2 Action-Level Verification on LIBERO-10

We use SmolVLA [33] to perform tasks on LIBERO-10 [24]. The verification module operates between the policy action queue and the environment step. It checks every low-level action online, and only committed Effect Receipts can update task state. Base and RoboGuardian use the same reinference frequency and differ only in whether the Commit Gate is enabled. The evaluation includes 10 tasks, 10 initial states, 3 conditions, and 2 variants, giving 600 episodes in total. We inject outdated action chunks and timeout and retry faults at a prevalence of 0.5. Execution stalls are evaluated separately as a secondary condition.

Under fault-free execution, Base and RoboGuardian achieve VSR values of 0.43 and 0.42, showing negligible capability loss from online verification. This comparable performance separates later fault-handling gains from changes in the underlying policy. RoboGuardian enters recovery in 34 episodes and resolves 7, showing that policy-level failures can still occur without injected faults.

Under outdated action chunks, both variants reach TSR 0.41, but Base verifies only 20 successes and produces an unverified write in all 50 exposed episodes. RoboGuardian verifies all 41 successes with no unverified writes. It resolves 25 of 67 recovery cases and reduces the mean action count from 426.8 to 378.5. The key gain is the separation of supported and unsupported success despite identical upstream TSR.

Under timeout and retry faults, Base duplicates the action in all 50 exposed episodes and verifies only 22 of its 42 successes. RoboGuardian re-observes before retrying, produces no duplicate actions or unverified writes, and verifies all 39 successes. Its TSR of 0.39 is slightly below the 0.42 of Base because it abstains when

Table 4: Clutter-Bench results over 63 paired scenes per condition. Each pair shares the target, clutter level, scene, random seed, and fault schedule. The pooled row is descriptive.

Condition	Variant	TSR ↑	VSR ↑
Clean	Base	0.857	0.825
Clean	RoboGuardian	**0.905**	**0.857**
False success	Base	0.476	0.460
False success	RoboGuardian	**0.889**	**0.889**
Response loss	Base	0.841	0.825
Response loss	RoboGuardian	**0.889**	**0.873**
Pooled	Base	0.725	0.704
Pooled	RoboGuardian	**0.894**	**0.873**

evidence remains unresolved. This tradeoff avoids treating an uncertain action as confirmed success or failure.

Under execution stalls, RoboGuardian enters recovery in 67.5% of eligible episodes and resolves 40.7%. Its VSR of 0.450 is close to Base at 0.425, so this condition represents a capability boundary rather than a gain. The framework detects a stall and invokes recovery, but cannot supply a recovery skill missing from the library.

4.3 Skill-Level Verification and Memory Governance

Clutter-Bench [4] provides high-level `push`, `pull`, and `grasp` skills over seven categories. We use clutter levels with 2, 4, and 6 distractors. RoboGuardian adds verification and receipt-gated memory at the skill boundary while retaining the environment and skills. False success fabricates a positive return without the intended effect, while response loss removes the return after a skill may have executed. We evaluate the three conditions over 63 paired scenes each. Each pair shares the target, clutter level, scene, random seed, and fault schedule. The memory ablation uses 30 scenes per condition and is not directly comparable with the main evaluation.

Under fault-free execution, RoboGuardian improves TSR from 0.857 to 0.905 and VSR from 0.825 to 0.857, indicating that skill-level verification preserves normal task capability. Base produces 72 unverified writes across 31 episodes, while RoboGuardian produces none. These writes arise from naturally occurring planning, perception, or skill errors because no external fault is injected.

Under false success, Base accepts the fabricated result and falls to TSR 0.476 and VSR 0.460. The planner then treats the grasp subgoal as complete and continues from a false premise. RoboGuardian rejects the result, returns control to the planner for another attempt, and reaches 0.889 on both metrics. Verification therefore supports recovery in addition to certifying outcomes.

Under response loss, RoboGuardian reaches TSR 0.889 and VSR 0.873, compared with 0.841 and 0.825 for Base. Base produces 32 duplicate actions and 49 unverified writes, while RoboGuardian produces neither by re-observing before another action is issued. Across all conditions, unverified writes fall from 170 in 102 of 189 episodes to zero, showing that trusted memory remains protected under failed and unresolved outcomes.

Figure 4 compares Ours-Window with the receipt-gated CGEM variant. Verified successes increase from 7, 1, and 7 to 23, 15, and 22 across the three conditions. Pooled VSR rises from 15/90 to 60/90, while mean skill dispatches fall from 51.6 to 35.0. Unlike a recency window, receipt-gated memory retains established effects in a compact form and reduces repeated subgoals.

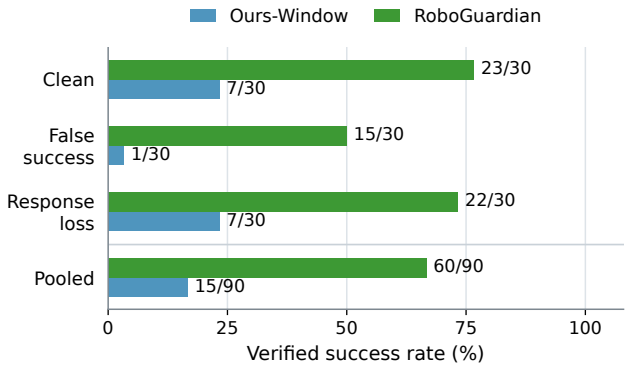


Figure 4: Memory ablation on Clutter-Bench. Bars show VSR. Labels show verified successes among 30 paired scenes per condition and among 90 scenes in the pooled result.

Table 5: Context and execution cost on Clutter-Bench. Actions and model calls are per-episode means. Max images is the largest number of images in any single model request.

Condition	Variant	Actions ↓	Calls ↓	Wall (s) ↓	Max images ↓	Peak request (MB) ↓
Clean	Base	21.0	21.9	654.4	53	21.21
Clean	RoboGuardian	**9.7**	**14.8**	**483.4**	**5**	**2.05**
False success	Base	15.4	16.4	1573.8	42	16.89
False success	RoboGuardian	**9.6**	**12.4**	**696.3**	**5**	**2.04**
Response loss	Base	22.2	23.1	1983.0	53	21.29
Response loss	RoboGuardian	**7.9**	**10.6**	**536.8**	**5**	**2.05**
Pooled	Base	19.5	20.4	1403.7	53	21.29
Pooled	RoboGuardian	**9.1**	**12.6**	**572.2**	**5**	**2.05**

Table 5 shows that RoboGuardian reduces the request ceiling from 53 to five images and from about 21.3 MB to 2.05 MB, which is close to an order of magnitude. In the pooled results, skill dispatches fall from 19.5 to 9.1, model calls from 20.4 to 12.6, and wall-clock time from 1403.7 s to 572.2 s. The bounded context therefore limits request size while reducing repeated planning and execution.

4.4 Cross-Modal Transfer to VLN

We integrate RoboGuardian with LaViRA [6] on the OpenNav-100 subset of Habitat R2R-CE [19, 34]. Navigation differs from manipulation in both horizon and action structure. A navigation episode contains a long sequence of motion decisions, while the native evaluator assigns outcomes to the complete trajectory. We therefore place the governance boundary around each macro-level evaluation rather than every low-level motion command. The original navigation stack and evaluator remain unchanged. The language-action model is GPT-4o, and the vision-action model is Qwen3.5-27B with reasoning output disabled.

We conduct three independent repeats with 100 episodes each, giving 300 evaluations in total. We report the metrics provided by the native navigation evaluator. Success measures whether the agent reaches the goal. SPL additionally accounts for path efficiency. nDTW measures agreement between the executed and reference

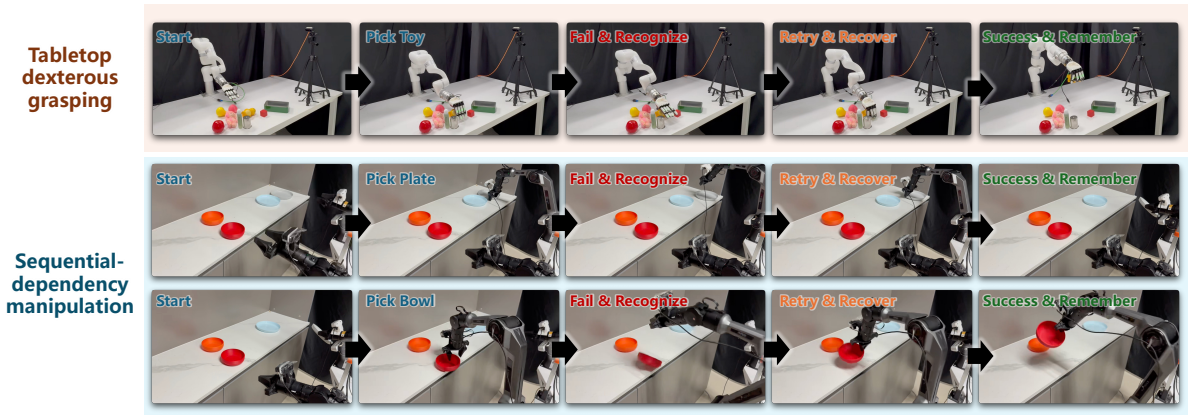


Figure 5: Qualitative experiments on the two physical robot platforms.

trajectories, while SDTW weights this agreement by task success. We do not report VSR because the macro-level integration does not produce a receipt chain for individual low-level actions. Reporting VSR in this setting would imply a verification granularity that the integration does not provide.

Across the three repeats, Success, SPL, nDTW, and SDTW are 0.327 ± 0.040, 0.245 ± 0.032, 0.427 ± 0.015, and 0.226 ± 0.037. The limited variation indicates that the integrated audit protocol produces stable evaluation records across repeated runs. More importantly, the same framework interface operates on navigation episodes even though their observations, action horizon, and task metrics differ from manipulation. This result supports the transfer claim in Q4 at the macro level. It does not show that RoboGuardian improves navigation performance or verifies individual motion commands.

4.5 Real-World Validation

The physical deployments evaluate Q2 and Q4 on two robot embodiments using evidence derived from real observations rather than simulator state. Fig. 5 summarizes the qualitative results. We examine whether the system rejects false software successes, enforces committed prerequisites for dependent skills, and records observations, judgments, and typed receipts for each run. Videos of the physical experiments are provided in the supplementary material.

Tabletop dexterous grasping. The first deployment uses a dexterous hand to execute a tabletop grasping sequence. Every skill is verified from camera observations captured before and after execution. Faults arise naturally and include grasp slippage, occlusion-induced detection errors, and skill returns that report success after hand closure without object retention. The qualitative video contains a false-success case in which the first `grasp` reports success even though the object is not held. Base accepts the return, writes the unverified effect into trusted state, and proceeds to `lift`. RoboGuardian instead assigns a negative judgment from the before and after evidence, rejects the receipt, and excludes the failed attempt from trusted memory. It then re-observes and commits only after a later grasp is verified. This case illustrates the difference between a software return and a physical fact that is eligible for commitment.

Sequential-dependency manipulation. The second deployment uses a dual-arm manipulator with grippers, checkpoint-backed high-level skills, and multi-camera observations. A later skill is admitted only while the earlier object-retention effect remains committed. This setting tests cases in which a committed prerequisite is invalidated by later slippage. In the first demonstration, the skill for the gray plate reports completion even though the robot does not hold the plate. Base treats the prerequisite as satisfied and dispatches the dependent skill for the red bowl, while RoboGuardian rejects the failed prerequisite and blocks the dependent step. In the second demonstration, the robot initially holds the gray plate and admits the red-bowl skill, but the earlier retention effect is later lost. Base accepts the final software completion despite the invalid prerequisite. RoboGuardian detects from goal-integrity evidence that the gray plate is no longer held and rejects the final result.

5 Conclusion

We introduce RoboGuardian, a framework for fault-aware execution and memory maintenance in embodied-agent harnesses. It distinguishes outdated plan steps, uncertain outcomes, and execution failures according to their required responses. Before dispatch, RoboGuardian checks plan validity and entity identity. After execution, it verifies physical effects from semantic, proprioceptive, and effect-level evidence, then converts the judgment into a typed receipt. Only committed receipts update verified memory and task progress. Rejected and unknown attempts remain available for diagnosis without altering trusted state. For unresolved outcomes, RoboGuardian re-observes before retrying and maintains a bounded context of verified evidence. Experiments in simulation and on physical robots show little overhead under fault-free execution. Under injected faults, RoboGuardian prevents unverified memory writes and duplicate actions while reducing the per-request context footprint by nearly an order of magnitude.

Verification remains limited by the available evidence, motivating broader physical evaluation under noisy perception. The re-observation limit must also balance safety and latency. This boundary between software reports and trusted physical effects becomes especially important in shared-state systems, where one unverified update can mislead every participating agent [15, 35].

References

[1] Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Chuyuan Fu, Keerthana Gopalakrishnan, Karol Hausman, et al. 2022. Do as i can, not as i say: Grounding language in robotic affordances. *arXiv preprint arXiv:2204.01691* (2022).

[2] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. 2024. π_0: A Vision-Language-Action Flow Model for General Robot Control. *arXiv preprint arXiv:2410.24164* (2024).

[3] Irvin Steve Cardenas, Marcus Anthony Arnett, Natalie Catherine Yeo, Lucky Sah, and Jong-Hoon Kim. 2026. ROSClaw: An OpenClaw ROS 2 framework for agentic robot control and interaction. *arXiv preprint arXiv:2603.26997* (2026).

[4] Zixuan Chen, Wenquan Zhang, Jing Fang, Ruiming Zeng, Zhixuan Xu, Yiwen Hou, Xinke Wang, Jieqi Shi, Jing Huo, and Yang Gao. 2026. AdaClearGrasp: Learning Adaptive Clearing for Zero-Shot Robust Dexterous Grasping in Densely Cluttered Environments. *arXiv preprint arXiv:2603.10616* (2026).

[5] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. 2025. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research* 44, 10-11 (2025), 1684–1704.

[6] Hongyu Ding, Ziming Xu, Yudong Fang, You Wu, Zixuan Chen, Jieqi Shi, Jing Huo, Yifan Zhang, and Yang Gao. 2025. LaViRA: Language-Vision-Robot Actions Translation for Zero-Shot Vision Language Navigation in Continuous Environments. *arXiv preprint arXiv:2510.19655* (2025).

[7] Danny Driess, Fei Xia, Mehdi SM Sajjadi, Corey Lynch, Aakanksha Chowdhery, Brian Ichter, Ayzaan Wahid, Jonathan Tompson, Quan Vuong, Tianhe Yu, et al. 2023. Palm-e: An embodied multimodal language model. *arXiv preprint arXiv:2303.03378* (2023).

[8] Yuqing Du, Ksenia Konyushkova, Misha Denil, Akhil Raju, Jessica Landon, Felix Hill, Nando De Freitas, and Serkan Cabi. 2023. Vision-language models as success detectors. *arXiv preprint arXiv:2303.07280* (2023).

[9] Jiafei Duan, Wilbert Pumacay, Nishanth Kumar, Yi Ru Wang, Shulin Tian, Wentao Yuan, Ranjay Krishna, Dieter Fox, Ajay Mandlekar, and Yijie Guo. 2024. Aha: A vision-language-model for detecting and reasoning over failures in robotic manipulation. *arXiv preprint arXiv:2410.00371* (2024).

[10] Yanjiang Guo, Yen-Jen Wang, Lihan Zha, and Jianyu Chen. 2023. Doremi: Grounding language model by detecting and recovering from plan-execution misalignment. *arXiv preprint arXiv:2307.00329* (2023).

[11] Wenlong Huang, Pieter Abbeel, Deepak Pathak, and Igor Mordatch. 2022. Language models as zero-shot planners: Extracting actionable knowledge for embodied agents. In *International conference on machine learning*. PMLR, 9118–9147.

[12] Wenlong Huang, Fei Xia, Dhruv Shah, Danny Driess, Andy Zeng, Yao Lu, Pete Florence, Igor Mordatch, Sergey Levine, Karol Hausman, et al. 2023. Grounded decoding: Guiding text generation with grounded models for embodied agents. *Advances in Neural Information Processing Systems* 36 (2023), 59636–59661.

[13] Wenlong Huang, Fei Xia, Ted Xiao, Harris Chan, Jacky Liang, Pete Florence, Andy Zeng, Jonathan Tompson, Igor Mordatch, Yevgen Chebotar, et al. 2022. Inner monologue: Embodied reasoning through planning with language models. *arXiv preprint arXiv:2207.05608* (2022).

[14] Xu Huang, Weixin Mao, Yinhao Li, Hua Chen, and Jiabao Zhao. 2026. Long-Term Memory for VLA-based Agents in Open-World Task Execution. *arXiv preprint arXiv:2604.15671* (2026).

[15] Dongjie Huo, Haoyun Liu, Guoqing Liu, Dekang Qi, Zhiming Sun, Maoguo Gao, Jianxin He, Yandan Yang, Xinyuan Chang, Feng Xiong, et al. 2026. ABot-Claw: A foundation for persistent, cooperative, and self-evolving robotic agents. *arXiv preprint arXiv:2604.10096* (2026).

[16] Aaron Hurst, Adam Lerer, Adam P Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, et al. 2024. Gpt-4o system card. *arXiv preprint arXiv:2410.21276* (2024).

[17] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, et al. 2024. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246* (2024).

[18] Alexander Kirillov, Eric Mintun, Nikhila Ravi, Hanzi Mao, Chloe Rolland, Laura Gustafson, Tete Xiao, Spencer Whitehead, Alexander C Berg, Wan-Yen Lo, et al. 2023. Segment anything. In *Proceedings of the IEEE/CVF international conference on computer vision*. 4015–4026.

[19] Jacob Krantz, Erik Wijmans, Arjun Majumdar, Dhruv Batra, and Stefan Lee. 2020. Beyond the nav-graph: Vision-and-language navigation in continuous environments. In *European Conference on Computer Vision*. Springer, 104–120.

[20] Chenxu Li, Zixuan Chen, Yetao Li, Jiapeng Xu, Hongyu Ding, Jieqi Shi, Jing Huo, and Yang Gao. 2026. MoMaStage: Skill-State Graph Guided Planning and Closed-Loop Execution for Long-Horizon Indoor Mobile Manipulation. In *RSS 2026 Workshop on Human-centric Mobile Manipulation*.

[21] Ruiying Li, Yunlang Zhou, YuYao Zhu, Kylin Chen, Jingyuan Wang, Sukai Wang, Kongtao Hu, Minhui Yu, Bowen Jiang, Zhan Su, et al. 2026. Roboclaw: An agentic framework for scalable long-horizon robotic tasks. *arXiv preprint arXiv:2603.11558* (2026).

[22] Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. 2023. Code as policies: Language model programs for embodied control. In *2023 IEEE International conference on robotics and automation (ICRA)*. IEEE, 9493–9500.

[23] Kevin Lin, Christopher Agia, Toki Migimatsu, Marco Pavone, and Jeannette Bohg. 2023. Text2motion: From natural language instructions to feasible plans. *Autonomous Robots* 47, 8 (2023), 1345–1365.

[24] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. 2023. Libero: Benchmarking knowledge transfer for lifelong robot learning. *Advances in Neural Information Processing Systems* 36 (2023), 44776–44791.

[25] Haowen Liu, Xirui Li, Shaoxiong Yao, Peng Shi, Tianyi Zhou, Jia-Bin Huang, Furong Huang, and Jiayuan Mao. 2026. Guava: An Effective and Universal Harness for Embodied Manipulation. *arXiv preprint arXiv:2606.18363* (2026).

[26] Songyang Liu, Chaozhuo Li, Chenxu Wang, Jinyu Hou, Zejian Chen, Litian Zhang, Zheng Liu, Qiwei Ye, Yiming Hei, Xi Zhang, et al. 2026. Clawkeeper: Comprehensive safety protection for openclaw agents through skills, plugins, and watchers. *arXiv preprint arXiv:2603.24414* (2026).

[27] Zeyi Liu, Arpit Bahety, and Shuran Song. 2023. Reflect: Summarizing robot experiences for failure explanation and correction. *arXiv preprint arXiv:2306.15724* (2023).

[28] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, et al. 2024. Dinov2: Learning robust visual features without supervision. *Transactions on Machine Learning Research Journal* (2024).

[29] Charles Packer, Vivian Fang, Shishir_G Patil, Kevin Lin, Sarah Wooders, and Joseph_E Gonzalez. 2023. MemGPT: towards LLMs as operating systems. (2023).

[30] Joon Sung Park, Joseph O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. 2023. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th annual acm symposium on user interface software and technology*. 1–22.

[31] Krishan Rana, Jesse Haviland, Sourav Garg, Jad Abou-Chakra, Ian Reid, and Niko Suenderhauf. 2023. Sayplan: Grounding large language models using 3d scene graphs for scalable robot task planning. *arXiv preprint arXiv:2307.06135* (2023).

[32] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems* 36 (2023), 8634–8652.

[33] Mustafa Shukor, Dana Aubakirova, Francesco Capuano, Pepijn Kooijmans, Steven Palma, Adil Zouitine, Michel Aractingi, Caroline Pascal, Martino Russi, Andres Marafioti, et al. 2025. Smolvla: A vision-language-action model for affordable and efficient robotics. *arXiv preprint arXiv:2506.01844* (2025).

[34] Andrew Szot, Alexander Clegg, Eric Undersander, Erik Wijmans, Yili Zhao, John Turner, Noah Maestre, Mustafa Mukadam, Devendra Singh Chaplot, Oleksandr Maksymets, et al. 2021. Habitat 2.0: Training home assistants to rearrange their habitat. *Advances in neural information processing systems* 34 (2021), 251–266.

[35] Huajie Tan, Xiaoshuai Hao, Cheng Chi, Minglan Lin, Yaoxu Lyu, Mingyu Cao, Dong Liang, Zhuo Chen, Mengsi Lyu, Cheng Peng, et al. 2025. Roboos: A hierarchical embodied framework for cross-embodiment and multi-agent collaboration. *arXiv preprint arXiv:2505.03673* (2025).

[36] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2023. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291* (2023).

[37] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388* (2025).

[38] Shunyu Yao, Jeffrey Zhao, Dian Yu, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. 2022. React: Synergizing reasoning and acting in language models. In *NeurIPS 2022 Foundation Models for Decision Making Workshop*.

[39] Takuma Yoneda, Jiading Fang, Peng Li, Huanyu Zhang, Tianchong Jiang, Shengjie Lin, Ben Picker, David Yunis, Hongyuan Mei, and Matthew R Walter. 2024. Statler: State-maintaining language models for embodied reasoning. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 15083–15091.

[40] Tony Z Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. 2023. Learning fine-grained bimanual manipulation with low-cost hardware. *arXiv preprint arXiv:2304.13705* (2023).

[41] Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. 2024. Memorybank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 38. 19724–19731.

[42] Enshen Zhou, Qi Su, Cheng Chi, Zhizheng Zhang, Zhongyuan Wang, Tiejun Huang, Lu Sheng, and He Wang. 2025. Code-as-monitor: Constraint-aware visual programming for reactive and proactive robotic failure detection. In *Proceedings of the Computer Vision and Pattern Recognition Conference*. 6919–6929.

[43] Brianna Zitkovich, Tianhe Yu, Sichun Xu, Peng Xu, Ted Xiao, Fei Xia, Jialin Wu, Paul Wohlhart, Stefan Welker, Ayzaan Wahid, et al. 2023. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning*. PMLR, 2165–2183.